UML diagramm

Statements

All nodes of the graph should be connected by queues

All nodes should be inherited from BaseNode

Use thread pool executor for all app

BaseNode should have implementation of logger

Use "spawn" mp in python besides mp.ProcessPool (because nvidia is being used)

Processes

Classes inside main/ subprocess

Queue's / Data Transfer

Threads

yaml config

```
shared_nodes:
 - name: shared_infer
   class: YoloInfer
   mode: process    # or "thread", if you want threading
   input_from: [reader1, reader2]
   output_to: postprocess_shared
   config:
     model_path: /models/yolov8.pt
     conf_thresh: 0.5

 - name: postprocess_shared
   class: BBoxFilter
   mode: process
   input_from: shared_infer
   output_to: null
   config:
     min_area: 100

pipelines:
 - name: video_pipe
   nodes:
     - name: reader1
       class: VideoReader
       mode: thread
       input_from: null
       output_to: shared_infer
       config:
         source: /videos/cam1.mp4
         loop: true

 - name: alert_pipe
   nodes:
     - name: reader2
       class: AlertImageReader
       mode: thread
       input_from: null
       output_to: shared_infer
       config:
         folder: /alerts/

 - name: test_pipe
   nodes:
     - name: dummy_reader
       class: DummyReader
       mode: thread
       output_to: dummy_infer
       config:
         fps: 2

     - name: dummy_infer
       class: DummyInfer
       mode: process
       input_from: dummy_reader
       output_to: null
       config:
         label: "car"
```

in

class: Task Creator

Description
Takes yaml, returns task class

Operations:
+ **init**(config):None
@staticmethod
+ __createTask(): Task

Attributes
- config

out

Task

in

class PipelineFactory

Description

Operations:
@staticmethod
+ create_pipeline(task:Task): BasePipeline

Attributes:
- ...

out

BasePipeline

run

Graph execution

class BaseReader

Attributes:
- source: Any
- queue: Queue
- thread: Thread
- stop_event: Event

Operations:
+ __init__(source: Any, queue: Queue): None
+ start(): None
+ stop(): None
+ read_loop(): None

class BaseNode

Attributes:
- in_queue: QueueLike
- out_queue: QueueLike
- mem_cache: Any

Operations:
+ __init__(config, in_queue: QueueLike, out_queue:QueueLike): None
+ process(self) # implement node logic across the process func
+ run(self)
+ stop(self)
+ __clean_cache(self)

class BaseInference(BaseProcessModule)

Attributes:
- batch: int
- in_queue: QueueLike
- out_queue: QueueLike

Operations:
+ __init__(self, config, in_queue: QueueLike, out_queue:QueueLike, batch: int): None
+ from_config(config)
+ run(self)

class BaseNode

Attributes:
- in_queue: QueueLike
- out_queue: QueueLike
- mem_cache: Any

Operations:
+ __init__(config, in_queue: QueueLike, out_queue:QueueLike): None
+ process(self) # implement node logic across the process func
+ run(self)
+ stop(self)
+ __clean_cache(self)

class BaseFilter(BaseNode)

Attributes:
- config: Config | dict

Operations:
+ from_config(config):Filter
+ update(self, new_data: Any): None (?)
+ process(self)
+ run(self)

class DataBaseOperator(BaseNode, Singleton)

Attributes:
- conn_string: str (?)
- login: str (?)
- password: str (?)
- port: int (?)

Operations:
+ update(self, new_data): None
- in_queue: QueueLike
+ process(self)
+ run(self)

dataclass: NodeConfig

+ name: str
+ type: str
+ params: Dict[str, Any]
+ output_queue: str

dataclass: Pipeline

Attributes:
+ name: str
+ nodes: Dict[str, NodeConfig]

dataclass: Task

Attributes:
+ name: str
+ pipeline: Pipeline

class BasePipeline(BaseProcessModule)

Description

Atributes
- ready: bool
- __initialized_digraph: nx.DiGraph[Frame, BaseNode]
- task: Task

Operations
+ run(): None
+ __queue_init(): None
+ __build_graph(): nx.DiGraph[Frame, BaseNode]
+ __check_health_thread
@staticmethod
+ __node_factory()

class BaseProcessModule

Description

Attributes:
- in_queue: QueueLike
- out_queue: QueueLike
- task: Task
- stop_event: multiprocessing.Event
- _last_success_time: float
- _healthy: multiprocessing.Value[bool]
- HEALTH_CHECK_INTERVAL: int = 5
- HEALTH_TIMEOUT: int = 15

Operations:
+ __init__(self, config: dict, in_queue: QueueLike, out_queue: QueueLike)
+ run(self) → None
+ stop(self) → None
+ process(self, item) → Any ← must be overridden
+ _health_monitor(self) → None
+ is_healthy(self) → bool
+ from_task(config: Task) → BaseProcessModule (optional factory)

fuction node_factory

Description
Takes NodeConfig and returns BaseNode

Operations
+ createNode(NodeConfig): BaseNode

BaseNode

Attributes:
- in_queue: QueueLike
- out_queue: QueueLike
- mem_cache: (?)

Operations:
+ __init__(config, in_queue: QueueLike, out_queue:QueueLike): None
+ process(self) # implement node logic across the process func
+ run(self)
+ stop(self)
+ __clean_cache(self)

QueueModules

get(timeout=1): Object
get_nowait(): Object
put(timeout): Object
put_nowait(): Object

BaseReader

Attributes:
- source: Any
- queue: Queue
- thread: Thread
- stop_event: Event

Operations:
+ __init__(source: Any, queue: Queue): None
+ start(): None
+ stop(): None
+ read_loop(): None

ImageFolderReader(BaseReader)

Attributes:
folder_path: str

Operations:
+ __init__(folder_path: str, queue: Queue): None
+ read_loop(): None

VideoReader(BaseReader)

Attributes:
video_path: str

Operations:
+ __init__(video_path: str, queue: Queue): None
# read_loop(): None